FPGA Based Breakout (Brick Breaker) Game

Using Verilog HDL and VGA Interface

By

Muhammad Haseeb

1 Objective

The objective of this project is to design and implement a hardware-based architecture for the classic Breakout (Brick Breaker) game on an FPGA utilizing Verilog Hardware Description Language (HDL). The game renders in real-time onto a standard VGA monitor. Crucially, the system operates completely within pure hardware fabrics without relying on embedded soft-core microprocessors, external software execution, or operating systems.

User interaction is facilitated through physical push-buttons handling real-time input. This design demonstrates the integration of key digital system architectural concepts, including synchronous pixel clock generation, active-video signal timing, real-time bounding-box collision detection, and structural multi-module synchronization.

2 System Overview

The hardware architecture processes player physical inputs and updates object states (paddle coordinates, ball velocities, and brick visibility arrays) synchronously at the video frame refresh boundaries.

The modular structure is segmented into four primary functional units:

- **Top Module:** Manages structural interconnects, external I/O mappings, and hosts the clock-management logic needed to derive the 25 MHz pixel clock from the 100 MHz onboard oscillator.
- **Debounce Synchronizer:** Isolates asynchronous bounce behavior from physical buttons and stages the raw signals through a dual-stage flip-flop register to mitigate metastability.
- **VGA Controller:** Orchestrates line-by-line raster signals, tracking absolute beam positions via horizontal and vertical scan metrics while generating vertical blanking triggers.
- **Game Core:** Encapsulates the algorithmic behavior of the application, managing brick-state arrays, spatial coordinate update registers, bounce direction dynamics, and pixel-color classification.

During active video periods, the structural subsystem operates by intersecting the current beam tracking indices with the dynamic object coordinates, evaluating asset presence instantly to output proper digital colors to the monitor.

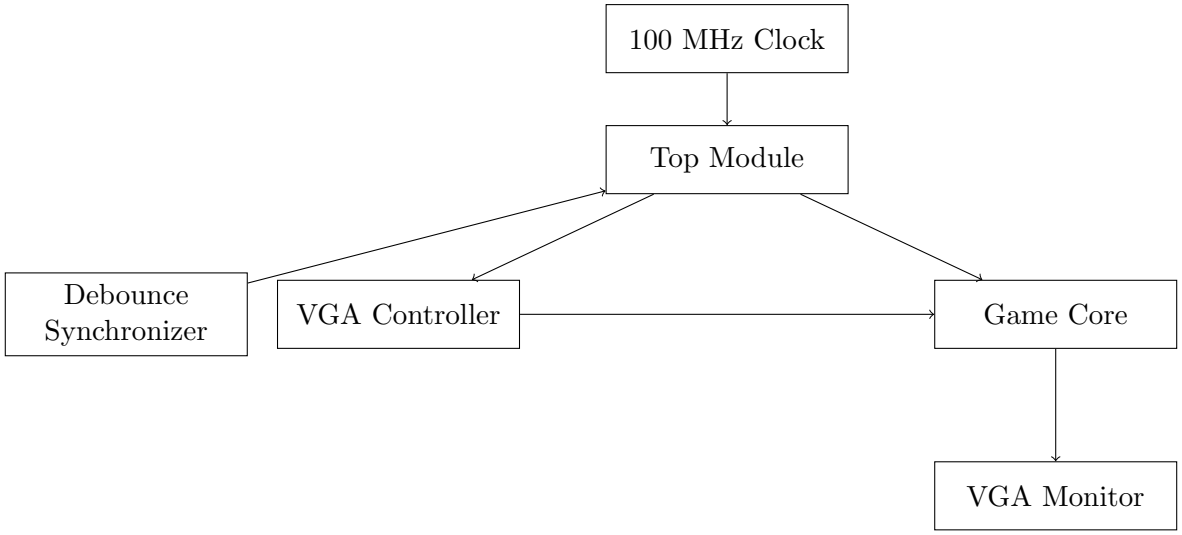

Figure 1: System Block Diagram and Interconnect Fabric

3 VGA Display Design

To accurately project the active assets, the VGA Controller acts as the timebase generator for the entire system. It establishes a target base display resolution of **640 × 480 pixels** clocked at a standard **60 Hz** refresh configuration.

3.1 VGA Timing Parameters

The sync generator maintains continuous monitoring counters driven by the local derived 25 MHz clock. Each display scanline includes non-visible styling offsets comprising front porches, backing sync anchors, and back porch segments. These formal baseline timing attributes are compiled in Table 1.

Table 1: VGA Timing Parameters

Parameter	**Value**
Active Video Resolution	640 × 480
Pixel Clock Frequency	25 MHz
Vertical Refresh Rate	60 Hz
Total Horizontal Lines Scan (Pixels)	800
Total Vertical Frame Scan (Lines)	525

3.2 Coordinate Tracking and Active Video Control

The system keeps precise tracking loops tracking active spatial coordinates (px and py). Color signals are explicitly governed by an internal `active_video` status flag. This logic

acts as a safety layer: when scanning lands inside blanking configurations or porch windows, the RGB pathways are clamped low to avoid synchronization distortions.

A dedicated singular pulse, `frame_tick`, is issued exactly on the arrival of each vertical blanking phase. By referencing state alterations directly to this vertical synchronization window, object step speeds are properly capped, creating fluid mechanics on the display.

4 Game Logic Design

The Game Core manages real-time physics tracking registers, boundary conditions, and spatial intersection equations.

4.1 Subsystem Entities and Behavioral Rules

- **Paddle Mechanics:** Movement relies on synchronous evaluation of debounced button states. Hard boundary clamps restrict the coordinate parameters, halting leftward or rightward shifts as soon as edge boundaries are detected.
- **Ball Vectors:** The position register updates via configured horizontal and vertical step values $(\Delta X, \Delta Y)$. Trajectory adjustments are accomplished by inverting directional polarities upon registered encounters.
- **Brick Array Grid:** A 2D status bitmask maps a dense wall of 50 individual bricks grouped across a 5×10 arrangement. Every element holds an operational bit flag; flags clear immediately when an intersection is detected, dropping the asset from subsequent drawing sweeps.

4.2 Spatial Collision Layer

Collisions are calculated by checking for geometric overlaps between the bounding boxes of the game assets. The system continually evaluates five boundary planes: the left, right, and top borders, the moving paddle surface, and active coordinates within the brick array grid.

When a collision registers against an active element within the brick matrix, the underlying tracking cell is toggled inactive, and the vertical velocity vector flips. Should the ball cross below the baseline threshold or if the bitmask tracks complete brick clearance, the state machine triggers a system-wide reset to restore the original coordinates and bitmask values.

4.3 System Execution Flowchart

The processing execution loop is outlined in Figure 2:

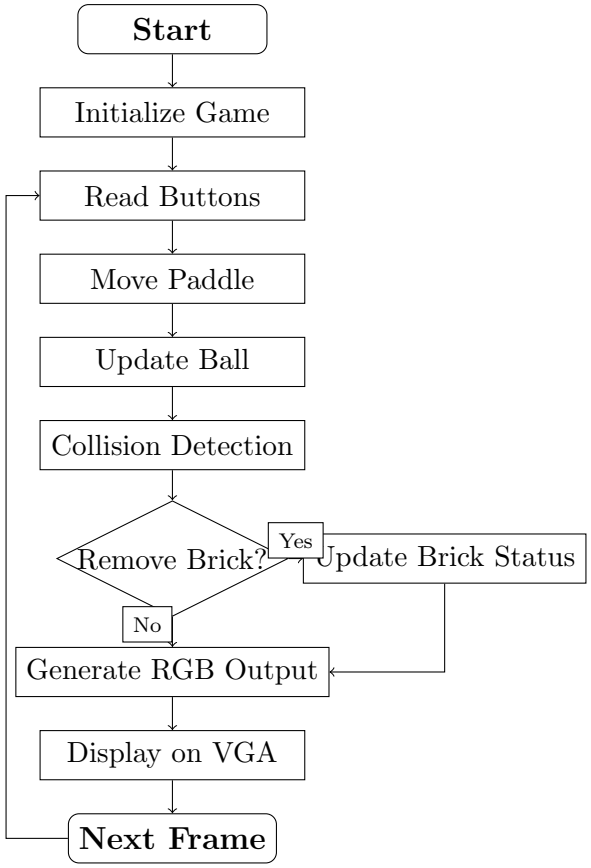

Figure 2: System Operational Loop and Execution Path

5 FPGA Implementation Results

Following behavioral verification, the source architecture was synthesized, mapped, and deployed onto a Mimas V2 FPGA development platform utilizing the Xilinx ISE toolchain. The compiled hardware configuration bitstream was successfully written to the board's non-volatile configuration memory.

Physical verification utilized a connected VGA monitor displaying active game objects. Interface responses tracked cleanly against physical push-button inputs. The system handled fast boundary transitions stably, managing the object tracking matrices and array resets without evidence of signal drops or screen artifacts.

A summary of the hardware implementation characteristics is detailed in Table 2.

Table 2: FPGA Hardware Configuration Matrix

Design Attribute	**Deployment Specification**
Target Evaluation Hardware	Mimas V2 FPGA Development Board
HDL Target Language	Verilog HDL
Development Software Environment	Xilinx ISE Design Suite
Display Interface Protocol	Video Graphics Array (VGA)
Target Output Dimensions	640×480 @ 60 Hz
Physical Input Hardware	Tactile Onboard Push-Buttons
Hardware Synthesis Evaluation	Closed Successfully / Fully Operational

6 Challenges and Solutions

Developing the game entirely in hardware required addressing several synchronization and timing challenges. Table 3 outlines the engineered resolutions implemented to ensure system stability.

Table 3: Engineering Challenges and Structural Solutions

Identified Challenge	**Engineered Solution**
VGA Raster Sync Variations	Fixed by binding standard horizontal and vertical counter limits directly to synchronous comparison networks.
Accelerated Asset Velocity	Implemented the `frame_tick` gating flag to restrict vector updates to vertical blanking intervals.
Physical Switch Contact Jitter	Added a dual-stage flip-flop shift pipeline to debounce raw inputs and prevent metastable transitions.
Overlapping Collision Boundaries	Developed sequential conditional checks using strict inequality operators to ensure clean directional rebounds.
Automated State Initialization	Implemented a dedicated reset routine to cleanly restore starting coordinates and re-enable the brick status matrix.

7 Conclusion

A hardware-only architecture for a classic Breakout game was successfully implemented on a Spartan-class FPGA utilizing Verilog HDL. By relying purely on synchronous digital logic circuits, the design achieves high-efficiency rendering without the overhead of a standard CPU or soft-core microcontroller.

The project successfully integrates several essential aspects of digital system design, including dual-axis video timing generation, debouncing networks, and real-time collision detection logic.

Potential future extensions include implementing multi-tone audio feedback generators, dynamic score counters, and scalable speed algorithms to support progressive difficulty levels.